

A laboratory method for AI-assisted science:
formal proof, exact arithmetic, controlled experiment, and a claim
ledger,
tested for a month on the Navier–Stokes problem

Socrate AI Lab*

18 September 2026

Abstract

Large language models produce mathematics and physics that read well and are sometimes wrong, and the discipline that has grown up around them is verification: proof assistants for proofs, and now public registries that check whether a repository proves what it claims. That discipline is necessary and it stops at the proof. A scientific laboratory also chooses questions, runs experiments, reads numbers, writes papers and publishes them, and none of those steps is certified by a kernel. We describe a working method that extends verifier-in-the-loop practice to the whole of that loop, and we report what it did over one month in two small laboratories working on the three-dimensional Navier–Stokes equations: one running the method in full on a machine-checked programme for the frequency-truncated system, the other auditing an AI-generated, Lean-certified blow-up proof and arriving independently at two of the same instruments. The method has four instruments — a tiered epistemic gate with formal proof at the top and exact rational arithmetic below it, a control discipline for every computational measurement, pre-registration of every prediction before the run that tests it, and a normative claim ledger outside of which nothing may be cited — and an agent framework in which model tiers, permissions and reusable skills are assigned by the cost of a wrong statement rather than by the difficulty of the task. The evidence is a record of failures: over the month, all but three of the recorded errors occurred with both mechanical gates (the kernel and the exact-arithmetic harness) green, and those were caught by a control, a pre-registered prediction, an external audit, or an adversarial review of a manuscript against its own formal sources; the three exceptions are the gate catching exactly what it is built to catch, and once catching too much. We list the incidents, what caught each one, and the rule each produced. The method does not make an AI laboratory correct. It makes its errors legible, dated, and cheap, which is what a laboratory needs. Nothing here is a claim about fluid mechanics or about the regularity of the Navier–Stokes equations; the two laboratories are the setting, not the result.

1 Introduction

In the last two years the question “can a machine do mathematics?” has been replaced by a harder one: “how do we know when it has?” Terence Tao’s account of machine-assisted proof [1] sets the

*A non-profit organisation for neuro-symbolic scientific AI. This manuscript was drafted by AI agents (Claude models) working under the adjudication of the laboratory’s human owner, and was audited before release by the same adversarial procedure it describes (see the Disclosure at the end).

frame: machine assistance is not new, but its scale and nature are changing, and the response of the community has been to make the machine’s output checkable — by a proof assistant whose kernel is small enough to trust, and now by a public registry, Palomar [2], that records a claim, compiles its Lean proof against a fixed library with only permitted axioms, and asks whether the formal statement is a fair rendering of the informal one. The Equational Theories Project [4] showed what that trust model makes possible: thousands of implications settled by contributors who did not need to trust each other, because every one was kernel-checked. In September 2026 an AI system produced Lean-certified proofs of finite-time blow-up for the forced Navier–Stokes equations and for the unforced Euler equations [5], and the discussion that followed turned, correctly, on what the formal statements say and whether they mean what the informal ones claim.

This paper is about the part of science that the kernel does not see. A laboratory does not only prove; it decides what to measure, builds an instrument, runs it, reads the number, decides what the number means, writes it up and publishes it. Every step is a place where an AI system — or a human — can be fluent and wrong, and only the first step has a mechanical gate. Tao’s own advice for the rest is a working rule rather than a tool: only rely on AI where you are able to red-team its output [3]. We take that rule as a design requirement and ask what a laboratory has to build so that the red-teaming happens by construction, before a result is load-bearing, and so that when it fails — as it will — the failure is recorded rather than absorbed.

Our answer is a method with four instruments and an agent framework, and our evidence is a month of running it. The instruments are:

1. a **tiered epistemic gate**: every claim carries a tier fixed by the strongest check it has passed — a kernel-checked proof with a clean axiom footprint, an identity verified in exact rational arithmetic with a control demonstrated to fail, or a conjecture — and a floating-point number can never rise above the lowest tier (Section 3);
2. a **control discipline** for computational measurement that treats the instrument, the control, and the code that decides “did this succeed” as claim-bearing artefacts needing their own evidence (Section 4);
3. **pre-registration** of every prediction, with its bracket and its factors, committed to version control before the run that tests it, and a rule that a falsified registration is reported as loudly as a confirmed one (Section 5);
4. a normative **claim ledger** and a **lessons file**: a claim absent from the ledger has no tier and may not be cited; a rule absent from the lessons file has no reason and will be shortcut (Section 6).

The agent framework (Section 8) assigns model tiers, permissions and reusable procedures — “skills” — to steps according to a single principle: *the cost of a wrong statement sets the tier*. Statement selection and audit sit at the top with the human; mechanical, gate-verifiable work sits at the bottom with the cheapest model; and the harness that runs the agents is itself an instrument that must be audited by something other than the agent it constrains.

The evidence (Section 9) is deliberately a record of what went wrong. Across the month, in both laboratories, the formal gates were green through nearly every substantive error; the exceptions are the cases the gate exists for. The rest were caught by controls, by pre-registered predictions, by an external audit of what a formal statement actually referred to, and by an adversarial review of a manuscript against the Lean statements it cited. We list them with what caught each and what rule it produced. We then state the method as a protocol (Section 10), in the spirit of Tao’s proposals but extended from proofs to experiments and to the boundary between a mathematical

model and the physical system it is a model of, and we close with what the month did not establish (Section 11).

Scope. Nothing here is a claim about fluid mechanics. The mathematical results of the first laboratory are published separately with their own audit [15]; the physical reading produced by the second is published with its own changelog of withdrawn claims [16]. This paper cites both only as evidence about method. The open problem both laboratories work near — global regularity of the three-dimensional Navier–Stokes equations — remains open, and nothing in either laboratory bears on it.

2 Related practice

Formal proof as the trust layer. Flyspeck [6] established that a proof too long for referees can be made trustworthy by a kernel; Buzzard’s survey [7] describes how proof assistants and language models began to meet; Tao [1] argues that the change is in scale and in the trust model, and the Equational Theories Project [4] is the existence proof: an implication graph on 4,694 laws settled by mixed human and automated proofs, all validated in Lean. Palomar [2] makes the trust layer a public service with two automated checks — a mechanical one, that the proof typechecks and proves the recorded statement against a fixed library with permitted axioms only; and a non-deterministic one, in which a language model judges whether the informal description in the submission’s disclosure file matches the formal claim and the repository meets minimal standards — and its announcement states plainly that these checks fall well short of peer review. Our tier system (Section 3) implements the first check locally and departs from the second: we reserve statement adequacy to human audit, for reasons the evidence section makes concrete.

Search with a verifier in the loop. AlphaProof [8] learns to prove in a formal environment where every step is checked; FunSearch [9] pairs a language model with a systematic evaluator to search program space and found new constructions in extremal combinatorics. Both rely on the evaluator being incorruptible and cheap. Karpathy’s *autoresearch* [10] applies the same shape to engineering: an agent proposes one change at a time, an immutable evaluator scores it under a fixed budget, and a mechanical rule keeps or discards. Section 8 adapts that loop to a laboratory, with the changes the control discipline forces on it.

Pre-registration. Nosek and colleagues [11] argue that registering a prediction before the data exist is what distinguishes a test from a fit. Computational science has been slow to adopt it because a simulation can be re-run; we found that this is exactly why it is needed there (Section 5).

Rigorous numerics. Where a computation is meant to be part of a proof, the standard is interval arithmetic with a self-consistent a-priori bound, as in Zgliczyński–Mischaikow [12]. Our exact-rational tier is the discrete analogue for identities, and our experience with iterating nonlinear maps in exact arithmetic (Section 7) is a caution about where the analogue stops.

3 The tiered gate

Every claim in the laboratory carries one of four tiers, fixed by the strongest mechanical check it has passed, and enforced by one script that either exits zero or does not. The formal tier is Lean 4 [13] against a pinned revision of Mathlib [14].

Tier	Meaning	Gate
A	Established: a Lean 4 theorem, kernel-compiled against a pinned Mathlib revision.	Zero <code>sorry</code> ; the compiled theorem’s axiom footprint, printed by the kernel, names no axiom outside <code>{propext, Classical.choice, Quot.sound}</code> . The gate is the footprint on the compiled artefact, not the absence of the word <code>sorry</code> in the source.
B	Checkable: an identity or inequality verified in exact rational arithmetic on concrete instances; a certified witness.	A harness using integers and rationals only — floating point is banned from the test directory — that exits zero, and that ships a negative control demonstrated to fail and, for any harness returning a verdict rather than asserting an identity, a positive control demonstrated to fire.
L	Literature: a published theorem, cited to its statement with the hypotheses that scope it.	A citation helper that refuses a numerical threshold lacking either its source or its hypotheses. A <code>L</code> row may motivate, scope or refute; it never discharges a formal obligation.
C	Conjecture: analogy, physical narrative, unverified reduction, and every floating-point computation.	None, but the tag is mandatory, and <code>C</code> code lives in a quarantined directory that nothing above it imports.

Three features of this table carry most of its value, and each was learned by getting it wrong.

The gate is a membership test on the compiled artefact. A failed Lean proof still defines its theorem name in the environment; only the printed footprint reveals the hidden `sorryAx`. An external submission to the first laboratory claimed four theorems “with no `sorry` placeholders”; compiled as received, one carried a custom axiom and one did not compile at all, and its environment carried `sorryAx` with no literal `sorry` in the file. The rule that followed — archive every submission verbatim, compile it exactly as received, save the kernel log as the evidentiary record — is the same rule Palomar implements as its first check. The gate itself was also wrong once: it string-matched the exact three-axiom list, so a lemma proved by pure computation, whose footprint was the strictly cleaner `[propext]`, was rejected. That false positive is not cosmetic. The cheapest way to satisfy an equality gate is to *add* an unnecessary axiom, so the gate would have trained proofs to get worse in the one dimension it measures. It is now a membership test, and the change — a loosening of a verification gate, exactly where a plausible edit can silently stop catching things — was accepted only after a six-case table including cases that must fail.

The verifier is pinned. The first cold build of the Lean project revealed that its dependency file required Mathlib from git with no revision. A fresh clone would have resolved to master and verified every ledger claim against a different library than the one they were checked with. For a programme whose entire epistemic claim is machine-checkability, an unpinned prover is a defect in the foundation. The library is pinned by revision and the lockfile is tracked.

Tier A attaches to the formal statement, never to its gloss. This is what the laboratory’s specification calls the honesty clause, and it is where the method departs from a registry that lets a language model judge statement adequacy. The largest failure in the first laboratory’s history was not a proof error. For weeks its specification, its report and its audit packet asserted that its formalisation “restates the open core of the Millennium problem”. An external expert audit found that it did not: a one-dimensional index cutoff, a weight 4^n and an unconstrained interaction term describe a dyadic shell hierarchy, not the three-dimensional equations. Two machine gates, five governing documents and an adversarial self-review had all passed, because none of them can check whether a formal statement refers to the object it names. The rule is that adequacy of a statement is certified by human audit sought early and from outside, and that a file marked DRAFT pending that audit may not have its gloss repeated in prose as if the audit had passed. A registry’s language-model check is a useful filter; it is not this.

4 The control discipline

A computational measurement has three parts that can be wrong independently of the science: the instrument, the control that is supposed to prove the instrument works, and the code that decides whether the run succeeded. The method treats all three as claim-bearing artefacts that need their own evidence. The rules below were each written after an incident, and the incidents are in Section 9.

A checker that cannot fail is not a checker — and that is half the requirement. Every harness ships a negative control: an input on which it must fail, and the failure is demonstrated, not assumed. The other half was learned when a structure detector shipped with an inverted sign and passed its negative control cleanly — generic inputs read $O(1)$ under both the correct and the inverted formula. Only a *positive* control, an input carrying the structure by construction and required to read zero, exposed it. Inverted, it would have reported “no structure” everywhere, including where the structure existed: a false negative indistinguishable from a clean null. Identity verifiers satisfy the positive direction by construction (the main sweep asserts the identity holds); classifiers and detectors do not, and for them an explicit positive control is mandatory.

Four ways a passing control is still worthless. A control tests the code against the premise the author believes. It cannot test the belief. The laboratory’s own catalogue:

- *Wrong premise.* A regime classifier’s positive-control anchor was a published threshold quoted from an abstract; the theorem statement, retrieved later, placed the threshold elsewhere and carried a positivity hypothesis the abstract did not mention. Both controls fired for a day on a wrong verdict. Rule: a numerical threshold is cited to the theorem statement with its hypotheses, never to the abstract, and a regression control pins the value that was wrong so the old belief cannot return.
- *Violated hypothesis.* A theorem-backed control failed because the run’s initial data violated the theorem’s hypothesis (infinite enstrophy in the limit), not because the code was wrong. Rule: encode the hypothesis as a runtime refusal, not a comment.
- *Ambiguous instrument state.* “Stopped early” meant both “the solution is blowing up” and “the step budget ran out”, and a summary read the second as the first in a regime where blow-up is

a theorem to be impossible. Rule: distinct stop reasons get distinct names, and an aggregate containing a resource-limit stop refuses to be interpreted.

- *Undemonstrated perturbation.* A negative control’s perturbation looked destructive and preserved the symmetry under test. Rule: a negative control is a claim carrying the same burden of proof as any other; and a failed negative control is often informative — this one became a better positive control.

Two corollaries do most of the work in practice. Prefer a control whose expected answer is a *published theorem*: it cannot be argued with, and in the August cycle every real catch by a control came from one. And when a control fails, the first hypothesis is that the control or the run is wrong — not the code, and never the world.

An instrument must report its own reliability. A fit, an inversion or a solve must report its conditioning alongside its value and refuse to report a value when the computation is ill-posed. The incident: a detector produced a clean, monotone, step-size-convergent time series confirming the hypothesis the laboratory existed to find; the fit behind it was singular at early times and the detector was dividing noise by noise. Convergence under refinement does not help — a singular fit converges to the same garbage at every step size. The rule that survives is: be most suspicious of the result you wanted.

The code that decides “did this succeed” is an instrument too. A month’s most expensive computation — an exact evaluation on rented hardware — converged correctly and was then declared failed twice by its own verification code: once because a plain text search treated a log containing a stray null byte (an artefact the log’s own unflushed design can produce) as binary and silently matched nothing; once because a resume path read a field that a *completed* checkpoint never writes. Both times the policy “discard as corrupt, restart from scratch” fired on a byte-for-byte valid checkpoint, and the second time it began a sixteen-and-a-half-hour recompute on a paid instance. Rules: success-detection code gets a negative control of its own, exercised on every state the process can be resumed into, including “already finished”; “discard” needs proof that the *data* are corrupt, obtained by reading the artefact with a different tool than the one that just failed on it; and a result that is independently backed up is *recovered*, not re-earned. The corrected binary, run locally against the mirrored checkpoint, reproduced the identical result in under a second.

5 Pre-registration, and reading numbers

The laboratory adopted pre-registration for computational measurements after reading a three-point series wrongly. The series was reported as “decelerating, consistent with saturation” from its successive ratios ($\times 5.3$, $\times 2.2$). Its differences were growing ($+0.0099$, $+0.0143$). Fitting the generic approach-to-a-ceiling form returned an inadmissible exponent: no saturating curve passes through the points at all. The reassuring reading came entirely from choosing one of two summaries of the same three numbers.

The registration procedure that followed has five steps and one non-negotiable property. State the rival hypotheses as parameterised functional forms, not as an observable (any number answers “does it grow?”). Fit every form to the points in hand and report inadmissible parameters. Report the trend in the units the claim is about — ratios and differences, side by side — and look for a bookkeeping mechanism that would produce the trend with none of the physics (a fraction climbing

to a hard ceiling did most of the “deceleration” here). Predict the next point with a bracket, by two routes where possible, and *pre-register the factors of a product rather than the composite*: an observable that is a product of separately measurable factors loses exactly the information that makes the prediction worth making when its composite is extrapolated. Then check that the prediction can discriminate: evaluate every rival at the point about to be run, and if the predictions overlap, say so before spending the machine. The non-negotiable property is the commit timestamp: the registration is written into the governing memo and committed before the run launches. A prediction written after the result and presented as registered destroys the entire value of the practice.

What this delivered in the month was not correct predictions. Of three registered brackets on one series, two were falsified — the first below the bracket, the second above — and the third held. Each falsification localised to one named factor of the product (once the injection, once the survival fraction), so the post-mortem produced a better model instead of an argument. The lesson file records the sequel honestly: on this family, no three-point extrapolation survived the next doubling, three times running, and the admissibility test that seemed decisive on three points reversed sign on four and again on five. An exact fit to as many points as parameters carries almost no information about the next point; report it with its parameter count attached, and never let it settle a question one more point can reverse. A falsified pre-registration is a success of the method and must be reported as loudly as a confirmed one — louder, since the confirmed ones are the ones nobody checks.

6 The ledger, the lessons file, and context as an artefact

Two documents make the method hold across many sessions of many agents, none of which remembers the last.

The **ledger** is normative: a claim not listed in it has no tier and may not be cited. It is checked mechanically on every verification run — every repository path it cites must exist, every Lean name in a top-tier row must be declared, and no versioned duplicate files may have appeared — and the checker ships its own controls (a dangling path, a dangling name and a banned filename must each be rejected, and a clean case must be accepted). It found a real defect on its first run.

The **lessons file** records, for every process rule, the incident that produced it, with evidence, what it cost or nearly cost, and where the rule now lives. It exists because a rule whose reason is lost is a rule that will be shortcut the next time it is inconvenient. Its entries are the primary source for Section 9; the file has thirty-one of them after a month, and two syntheses — the four ways a passing control is worthless, and the three blind spots of a two-gate system (cost, interpretation, and the harness itself) — that were written when the entries had accumulated into a pattern.

Context, in this method, is not held in any agent’s memory. It is externalised into five documents with a strict precedence — the specification, the operational plan, the ledger, the lessons, and the build manual — and every session loads them before acting. The report of the first laboratory places this instruction first, because acting without it had already caused documented failures.

7 Mathematics and physics: the model, the simulation, and the boundary

A laboratory that does both proof and experiment needs a stated relation between them, and the month produced three rules.

Floating point is Tier C, always. Simulation is indispensable for steering and inadmissible as evidence. The scout code that measures transients in the truncated system was calibrated against eight steps of an exact rational integrator to 2.4×10^{-16} before any of its numbers were read, and every one of its results carries the lowest tier no matter how clean it looks. The exact integrator itself could not go further: iterating a nonlinear map in exact arithmetic is a representation-size problem, not a stability problem — denominators reached tens of thousands of bits within a handful of steps while the magnitudes stayed modest — and a second numerical design that fixed the stiffness diagnosed from the first failed anyway, because it had fixed the wrong axis. The rule: exact arithmetic for one-shot algebraic certification, floating point for trajectories, and never confuse a divergence in representation size with a divergence in the quantity.

Physical narrative is quarantined. The motivating physics of the first laboratory lives in a directory that the formal sources never import and that no tier promotion may cite. The sense of analogy that motivates a programme is welcome there; it earns its way into the core only through the exact and formal tiers. This is a stronger separation than most programmes keep, and it was tested: at least four proposed physical mechanisms were killed on measured evidence within the month, and none of the kills required touching a theorem.

A theorem about a model is labelled, not rejected. The second laboratory’s subject is a Lean-certified proof that a forced incompressible fluid blows up in finite time. Its current position, after a sequence of withdrawn stronger claims, is that the constructed flow leaves the validity range of the incompressible continuum model — at the single scale where Mach, Knudsen and Eckert numbers all become order one — a few picoseconds before the mathematical singularity for water, nanoseconds for air. The formal counterpart is proved in Lean on the original proof’s own definitions: every object with the construction’s properties exceeds every velocity-gradient bound arbitrarily close to the singular time. The methodological point is the framing the laboratory arrived at after discarding its first one: a physical-validity layer should *label* a result as a theorem about a model rather than a statement about a fluid; it should not reject a correct proof. Its early manifesto proposed a “**physlib**” world model under which “the AI will automatically discard candidate proofs” that are physically implausible, and its current status note withdraws that phrase. The proof is correct. The question of which model it is about is a scientific question, answered by a predicate — here a bound on local vorticity equivalent to Mach and Knudsen bounds together — that is stated, formalised where it can be, and reported with the theorem.

The same laboratory states the discipline in one sentence its programme document uses as a design requirement: “a lock without a test is a slogan.” Every proposed lock — its term for a physical mechanism that would arrest the collapse — is written with what it is, whether it is derived or posited, what it predicts, and what could be formalised. And it applied that standard to its own proposed remedy, a cutoff regularisation, and found that the remedy could not be evaluated at all with generic data — “it is easier to identify where a model fails than to establish that a proposed repair does anything,” its paper records, “and the standard of evidence we applied to [the original]

construction is the one we then had to apply to our own suggestion.”

8 The agent framework

8.1 Roles by the cost of a wrong statement

The framework assigns work to model tiers and to the human by one principle: the cost of a wrong statement sets the tier. In the first laboratory’s adjudicated workflow:

- the **human owner** supplies definitions and verdicts — what is to be proved, whether a formal statement is adequate, whether a result is accepted, where doubt and compute are spent — and never the content of a proof or a script;
- the **top model tier** writes derivations, statements and audits: the hand derivation and the fully specified proof skeleton that precede any dispatch, the adversarial review of a claim or a manuscript, the escalation when a task needs a definition that does not exist;
- the **middle tier** implements from fully specified memos — a proof against confirmed-available lemmas, a harness against a stated identity — and is not asked to decide what to prove;
- the **cheapest tier** does gate-verifiable mechanics, where a wrong output is caught by the next gate at no cost.

The order is never reversed to save a step. Every proof that closed on the first or second attempt in the month was preceded by a hand derivation, a numeric instance, and a check that the named library lemmas were actually built; tasks that asked an agent to *choose* the statement either escalated correctly or needed a human decision partway through.

8.2 The screen: autoresearch with controls

When several plausible variants of a design exist and no principled reason prefers one, the framework runs a fixed-budget screen after Karpathy’s pattern [10], with three changes the control discipline forces. First, the hypotheses are proposed by several independent agents of a cheaper tier, each given the same facts and a different lens (optimisation, cost, controls), *without access to the code*; a stronger-tier judge synthesises the final list, the control rows, and a mechanical decision rule, and all of it is committed before the first row runs. Second — the step that caught the real bugs — the authoring session checks every proposal against the code path it presumes to describe before the design becomes binding. In the month’s one screen this found two proposals that assumed a mechanism the code provably does not have, and a genuine convergence bug in the optimiser being screened, visible only because a proposal’s assumption prompted a direct read of the loop it reasoned about. A panel review is a review of the description, not of the code. Third, the loop does not rewrite its own hypotheses: when a registered rule misfired on real results, it was run as registered, both readings were reported, and the alternative was added as an explicitly post-hoc row that could not win. Changing the rule after seeing the table is the thing pre-registration exists to prevent; hiding the misfire is worse.

8.3 The harness is an instrument, audited from outside

Three rules concern the machinery that runs the agents rather than the agents.

A permission boundary is enforced by the boundary. A session denied the ability to edit its own permission file observed that an allowed general-purpose file-rewriting command would have let it rewrite that file regardless, and reported the denial instead of using the opening. The rule is not “agents should be restrained”; it is that an allow-list containing a general rewriter does not deny anything, and must be audited for rules that subsume the denials — before granting unattended overnight compute, which is when nobody is watching. The same session, during a later overnight run, had a cloud-provisioning tool remain denied for the run’s whole duration despite the owner adding it to the allow-list, because the settings watcher does not reload without a restart; it reported the limit twice rather than working around it.

Blast radius is scoped. Any command whose effect is the whole working tree — a broad `git add`, a `stash`, a `reset` — is forbidden while any concurrent process may be writing. A broad `add` twice swept a mid-edit file into a commit; the second time it was a proof, and the commit message, written before the concurrent state was known, said “both gates pass” — true for the files checked and false for the one incidentally captured. Concurrent tools work in their own worktree and merge back through review; a file modified in the main checkout by anything other than its owning session is an escalation, not a merge.

Self-reports are not evidence. A tool’s own status field, a subagent’s final report, a submission’s “no sorry by construction”, the deposit script’s own `PUBLISHED` line: each is re-run or re-fetched on the exact artefact before it is believed. The month’s cases include a subagent whose final report would have said “done, compiles clean” — truthfully — and never mentioned the repository-wide `stash` it had run in the middle.

8.4 Skills and agents

The procedures above are packaged as reusable instruction sets — skills invoked by name, and agent definitions with restricted tools — so that a future session applies them without re-deriving them. Each encodes one failure. They are proposed in a documentation directory and installed by the owner, never by the agent that wrote them, for the reason in the previous subsection.

Artefact	Encodes	Origin
preregister	Rival hypotheses as forms; fit and admissibility; ratios and differences; factors not composites; discrimination check; committed before the run.	A three-point series read wrongly; a decisive point that was not decisive.
cost-model	Count, never estimate; explicit <code>sizeof</code> with padding and transients; $O(\cdot)$ model checked against measured scaling; optimisations verified as refactors; hardware asked for last.	A memory declaration $6.6\times$ optimistic; a quadratic optimiser correct for three campaigns; a provably-correct optimisation 5.5% slower.
resilient-job	Checkpoint inside every expensive pass, self-verifying and torn-write-safe; success detection with its own controls, on every resumable state; discard needs proof of corruption; recover the value, not the compute; three telemetry channels.	A converged cloud job declared failed twice by its own verifier; a needless recompute begun twice.
autoresearch-screen	Independent proposers without code access; every proposal checked against the code; judge fixes list and rule before any row; misfires reported as registered.	A panel’s design with a real bug in the software it screened.
publish-gate	Author of record supplied, never inferred; artefact matches the tag by checksum; gates pass at the tag; metadata read against the withdrawn list; draft first, publish separately; live record verified anonymously; exposed secrets rotated.	A paper one flag from a permanent DOI with a placeholder author; a push script carrying withdrawn claims.
claim-auditor (agent)	Adversarial audit of one claim against the three blind spots of a two-gate system: interpretation, cost, the harness. Read-only.	Every failure of one cycle occurring with both gates green.
manuscript-auditor (agent)	Every theorem diffed against its Lean signature; every number traced to its source file, not the ledger; every novelty claim against retrieved literature; scope, tone, hygiene. Read-only.	A first draft with two blocking defects, written by the session that had proved the results.

9 Evidence: two laboratories, one month

9.1 The first laboratory

The first laboratory began on 12 August 2026 and, at the date of this paper, holds 193 kernel-verified theorems in twelve Lean files (5,498 lines; 186 at the top tier, seven demoted by the audit of Section 3), twenty-two exact-arithmetic harnesses plus a shared control library, twenty-seven design memos and two decision records, nine filed escalations, thirty-one recorded lessons, 167 commits on thirteen active days, and eleven release tags. Its mathematical output is a six-page paper on exact triad structure in the truncated equations, archived with a DOI [15]. Its scientific target, a uniform entrophy bound on the truncated system, is open, and the report says so on every page that could be misread.

Table 1 lists the incidents that produced rules, with what caught each. The column that matters is the fourth. Of the twenty-two entries, nineteen occurred with both gates green and are not proof

errors. The three exceptions are the two `sorryAx` incidents, which the kernel gate did catch, and the gate’s own false positive.

Table 1: Incidents in the first laboratory, what caught them, and the rule each produced. `LL- $n$` is entry n of the laboratory’s lessons file, `LL.md`; a dash marks an incident recorded elsewhere in the repository.

LL	What went wrong	What caught it	Rule
—	The central framing claim (“restates the open core of the Millennium problem”) was false of the formalisation.	External expert audit.	Statement adequacy is human, early, outside; DRAFT files may not have their gloss repeated.
1, 9	Broad <code>git add</code> and a subagent’s <code>stash</code> swept concurrent edits; a commit carried <code>sorryAx</code> in five footprints under a “gates pass” message.	Luck, then an unprompted diff by the authoring agent.	Stage by explicit path; no tree-wide git command while anything may be writing.
2	A submission “proven, no sorry” did not compile; footprint had <code>sorryAx</code> .	Compiling the artefact as received.	Archive verbatim, compile as received, keep the kernel log.
3, 4	Two numerical designs failed for different reasons under the same symptom; the second fixed the wrong axis.	Inspecting representation size versus magnitude.	Diagnose the axis before redesigning; two failed designs is the threshold to escalate methodology.
7	A top-tier design memo’s own worked example used the wrong summation range.	The certifying agent reproducing the memo’s numbers before running.	Downstream re-verification is not removed by upstream authority.
8	The Tier A gate rejected a strictly cleaner footprint.	A lemma proved by computation.	Membership test; test a changed gate against cases that must fail; read the exit code.
—	Mathlib was unpinned.	Doing the cold build.	Pin the verifier; track the lockfile.
11	A theorem translated to a new setting lost its content: the constraint admitted every power law.	Computing the constraint’s solution set.	Check a translated constraint restricts the simplest family the new setting supports.
12	A detector shipped with an inverted sign; the negative control passed.	The positive control.	Classifiers need a demonstrated positive control.
13	Two campaigns measured boundedness in a regime where it is a published theorem.	Retrieving the theorem.	Classify the regime before measuring; a proven regime is a control, not a discovery.
14	A clean, monotone, convergent series confirming the hypothesis was a singular fit.	Asking whether the fit was well-posed.	Instruments report conditioning and refuse when ill-posed.
15	A pre-registered test passed and was undiagnostic: the effect appeared where it is provably impossible.	The theorem-backed control run alongside.	Every registration names a control where the effect must be absent.
16, 17, 18, 19	Wrong premise; violated hypothesis; ambiguous stop; undemonstrated perturbation (Section 4).	Theorem statement retrieved; a control failing; a theorem column showing blow-up; a control not failing.	The four requirements now enforced at runtime.

LL	What went wrong	What caught it	Rule
20, 24	“Decelerating” read off ratios while differences grew; a composite extrapolated instead of its factors; two registered brackets falsified.	Computing the differences; the registered bracket.	Units of the claim; fit and admissibility; factors not composites; falsifications reported loudly.
21	A declared memory risk was $6.6\times$ optimistic.	Counting entries and <code>sizeof</code> .	A declared risk carries a counted quantity.
22	An optimiser re-summed 1.35×10^8 terms per candidate for three campaigns.	The first run that took hours.	Correctness gates cannot see cost; state the $O(\cdot)$ model first.
23	The allow-list subsumed its own denial.	The agent, reporting instead of using it.	Boundaries enforced by the boundary; audit allow-lists.
25	An aggregate was read from a file still being written.	A monotonicity impossibility (refinement cannot lower a maximum).	Verify inputs complete; keep a list of impossibilities.
26	Four unattended jobs killed by desktop memory pressure they did not cause; 1.5 h lost.	Reading the resident sets.	Budget against the low-water mark; checkpoint before the incident that proves you should have.
27	A provably-correct, bit-identical optimisation was 5.5% slower.	A fair clock.	Theory bounds the win; only the clock proves it; keep the negative result.
28	A panel’s screen design assumed mechanisms the code lacks and contained a bug in the screened software; a registered rule misfired.	Reading the code against the proposals; running the rule as registered.	Panels review descriptions; check against code; report misfires.
29, 30, 31	Verification code declared a converged job failed twice; a valid checkpoint was quarantined and a recompute begun.	Reading raw bytes with a different tool.	Verifiers need negative controls; discard needs proof of corruption; recover the value.
—	A paper draft overclaimed novelty against a 1992 result and stated a theorem without a hypothesis its Lean proof requires.	Adversarial review against the Lean signatures and retrieved literature.	The manuscript auditor.
—	A paper was one flag from a permanent DOI with a placeholder author on every page and an inferred creator name.	The deposit script’s refusal to publish without a second flag.	The publish gate.

Two summary numbers from the lessons file are findings in their own right. In one cycle, three separate publishable-looking numbers were each caught as artefacts — a null-model catch, a mis-stated regime band, and a compute-limited sweep — none by review, all by pre-registered controls, two of which were themselves defective at first. And the lessons file’s own estimate, stated flatly at the head of its synthesis, is that in this domain the artefact rate for uncontrolled measurements is close to one.

9.2 The second laboratory

The second laboratory audits an AI-generated, Lean-certified proof [5]. Its current position [16] is that the proof is correct and that the constructed flow leaves the validity range of the incompressible

model before the singularity. At its archived version 5.6.1 it holds ten Lean files with 85 declarations with verified axiom footprints, one of them stated on the original proof’s own definitions; a reproducibility benchmark that re-runs every quoted number and compares it bit-for-bit with the committed value (54 checks at release 5.5.0); and a 31-page paper with a concept DOI. It is run by the same owner as the first laboratory, with no shared code or method. It did not adopt the first laboratory’s method. It converged independently on two of its instruments — the kernel gate with the same axiom footprint, and a changelog in which corrections are recorded rather than rewritten — and it made the opposite choice on a third: it has no exact-arithmetic tier, every simulation is floating point, and what stands in for exactness is bit-identical reproducibility on a pinned software stack, a weaker property that the benchmark states explicitly. Its enforced continuous-integration workflow is a stale legacy pipeline whose runs fail at startup; the current Lean check is run on demand. That makes it a useful second data point rather than a replication.

Its evidence for method is the changelog and the paper’s appendix on previous versions, which sorts what changed under six headings: claims withdrawn, material removed, citations corrected, reframed not retracted, withdrawn in a later version, and forecasts not borne out. Table 2 lists the entries that bear on method, with what caught each. Three deserve comment. A “Lighthill energy-budget check” had been presented as an independent confirmation of the Mach criterion; dimensional analysis showed the two were the same criterion, so the corroboration was a single check counted twice. A global enstrophy constant used as an admissibility axiom since early versions was withdrawn when someone asked where it came from and found it “had no derivation anywhere in the repository”; it had propagated through papers, code, a table generator and Lean docstrings. And an “interception theorem” whose file claimed “0 sorry” in prose turned out, on re-compilation, not to build against the current toolchain at all — and its statement was circular besides: its admissibility hypothesis contained the withdrawn constant, so at best it would have proved only that a divergent quantity is not bounded. Two rules in one artefact: a self-reported kernel check is not a kernel check, and a kernel, had it checked it, would not have noticed that the antecedent contained the conclusion.

Table 2: Withdrawn or corrected claims in the second laboratory, from its changelog and the appendix of its paper, with the mechanism that caught each.

Claim	Caught by	Recorded as
“Physical vacuity” and “censorship” of a correct proof.	Public community review; the project conceded that the theorem is about the model it names.	Framing withdrawn; “label, not reject” adopted.
Plasma temperatures at the core.	Internal re-derivation: $\Delta T \simeq u^2/c_p$ gives hundreds of kelvin, not 10^4 .	Withdrawn.
An acoustic energy budget as independent evidence.	Dimensional analysis: the budget is the Mach criterion in disguise.	Removed as evidence.
A raw 10^{28} condition number as fragility.	Non-dimensionalisation: $\kappa \approx 4 \times 10^5$ at every scale; double precision is unreliable past 10^{16} anyway.	Withdrawn.
A global enstrophy ceiling used as an axiom.	Provenance audit: no derivation anywhere.	Withdrawn everywhere it appeared.

Claim	Caught by	Recorded as
Femtosecond timings.	Unit audit: a dimensionless time read as seconds, twelve orders of magnitude.	Corrected; found the same day still live on a public dataset.
“Sub-Planckian” scales.	The relevant cutoff is the molecular mean free path.	Corrected; the residual claim downgraded to an open question.
A string-theoretic motivation.	Independent review of the earlier paper: the load-bearing Lean conjunct was a bare True ; an equation lacked a term.	Reframed, not retracted.
“Lemma 8.7”; “10,000 agents in 88 hours”.	Checked against the primary PDFs: the first is Proposition B.8; the second appears in neither paper.	Corrected; press figures no longer repeated as fact.
A hand-placed “pre-singularity spectrum”.	Internal review.	Figure removed; the surviving schematic labelled as such.
A “dissipative kinetic lock”.	An exact spectrum computed two independent ways, agreeing to 3×10^{-12} : the first kinetic correction reduces damping.	Withdrawn with a numbered correction.
“45 theorems” in Lean.	A counting-convention audit: 36 by the convention adopted.	Convention stated.
A 96^3 sweep would reach a threshold crossing.	Running it: 0 of 6.	Forecast not borne out.
A hypothesis named for a classical theorem it was weaker than.	Re-reading the hypothesis.	Corrected; then proved unconditionally.
A grid-refinement endpoint of 0.86λ .	The benchmark’s re-derivation check: wrong metric; 0.96λ .	Corrected.
A filter model at the validity scale represents the fluid.	A four-declaration Lean file: the filter parameter is absent from the linearised dynamics.	Refuted; given its own section.
A quasi-steady arrest at Mach 0.36.	A grid-converged compressible simulation: no run arrests.	Forecast not borne out.
A “ physlib ” world model that discards implausible proofs.	Re-compilation: the drafts carry a custom axiom and a True placeholder.	Withdrawn; no such tool exists.

The changelog also records two things the first laboratory’s method now borrows. On 15 September the project’s Zenodo upload script still carried the title, description and file list of the withdrawn framing; run unmodified, it would have re-published retracted claims under a permanent identifier. The same day the project found its public dataset already serving the retracted paper and the unit bug, live, under a real name. Both were caught by reading before running, and the release procedure that followed — every file’s checksum against the tagged commit, metadata read against the paper, one stale sentence caught in that very check — is what our publish gate encodes. And the project’s peer review, recorded verbatim with a point-by-point response, contains an editorial observation worth passing on: the reviewer asked that the manuscript’s running comparisons with “earlier drafts” be moved into an appendix — that transparency be *relocated*, not reduced — and the authors did so while adjudicating each of the reviewer’s four reported table defects individually: one

was an extraction artefact of the reviewer’s own copy, one a genuine defect, and two were rebuilt for robustness. The project also supplies one dated pre-registration across the model boundary. A continuum prediction that the local Mach number in a driven core would saturate near 0.6 was committed on 17 September, ten hours before the first molecular-dynamics data existed; the molecular simulation, with no viscosity law, heat-conduction law or equation of state put in, gave a fitted value of 0.538 ± 0.011 (and 0.61 by a second estimator) against a continuum value of 0.54 recomputed with the measured transport coefficients. The first published reading of that comparison overstated the molecular numbers and was itself corrected in the changelog — which is the discipline working, not failing: the registration is dated, and the correction is dated after it.

9.3 What the two laboratories show together

Three things, and the third is the finding of this paper.

First, the tiered gate does what it is for and no more. Across both laboratories, no theorem promoted to the top tier has been found wrong, and every attempt to promote a statement that was not adequate was caught — once by an external audit, once by the laboratory’s own honesty clause refusing to let a DRAFT gloss stand as a claim. The gate bounds what may be *claimed*.

Second, the gate is silent on what should be *asked*, what it should *cost*, and whether the *instrument* is sound. All but three entries in Table 1 occurred with both gates green. The instruments that caught them — controls, registered predictions, cost models, external audit, adversarial review — are cheap, mechanical, and none of them is clever. Their common form is the conversion of an assertion into a counted or measured quantity before it becomes load-bearing.

Two of the same errors occurred in both laboratories independently, which is the closest thing this paper has to a replication. Each quoted a Lean theorem count under one convention and later found it was another (“45 theorems” that were 36 by the convention adopted; a report’s 258 that the gate certifies as 193), and each fixed it the same way, by stating the convention and taking the number from the gate’s own transcript. And each carried a transcription error from an internal record into a manuscript — a “seven-point” series that has six points, a hand-placed figure labelled as computed — caught only by tracing the number past the record to the file that produced it. Counts drift with conventions and transcriptions drift with distance from the source; neither is a proof error, and both reached a manuscript.

Third, the method’s product is not correctness but *legibility*. Both laboratories were wrong repeatedly. The difference the method made is that each error is dated, localised to a named factor or a named line, attributed to the mechanism that caught it, and attached to a rule — and that the wrong reading was never available to be published, because the ledger had no row for it. A laboratory cannot promise to be right. It can promise that its errors will be this shape.

10 The proposal: a protocol for an AI-assisted laboratory

We state the method as rules, in the order a new laboratory would adopt them. Each is a consequence of an incident above; none is a preference.

1. **Tier every claim by the strongest check it has passed, and enforce the tier by a script that exits non-zero.** Formal proof with a clean footprint on the compiled artefact; exact arithmetic with demonstrated controls; literature cited to the theorem statement with its hypotheses; everything else, including all floating point, conjecture.

2. **Keep a normative ledger.** A claim not in it may not be cited. Check it mechanically for orphans on every run.
3. **Reserve statement adequacy to a human, sought early and from outside.** The kernel checks the proof; no gate checks that the statement refers to the object it names. A language-model check is a filter, not a certificate.
4. **Derive by hand before dispatching; verify downstream anyway.** The top tier writes the statement and the skeleton; the implementer executes; the certifier reproduces the memo’s own numbers before running the sweep — because the memo can be wrong.
5. **Every measurement ships both controls, and the controls carry their own burden of proof.** Negative and positive; premise cited to a theorem; hypotheses enforced at runtime; stop reasons named; perturbations demonstrated. Prefer a control whose expected answer is a published theorem.
6. **Instruments report their own reliability and refuse when ill-posed.** Be most suspicious of the result you wanted.
7. **Pre-register the prediction — as factors, with a bracket, with a control where the effect must be absent, with a discrimination check — and commit it before the run.** Report a falsification as loudly as a confirmation. Never present a post-diction as a registration.
8. **State the cost model before scaling; count, never estimate; report measured factors, not operation counts.** A correct answer is camouflage for a bad algorithm, and a rented machine conceals cost failures rather than exposing them.
9. **Treat verification code, checkpoint readers and success detectors as instruments.** Give them negative controls on every state they can encounter. Before discarding data, prove the data are corrupt with a different tool. Recover backed-up results rather than re-earning them.
10. **Quarantine physical narrative and floating point; label theorems about a model as such.** A physical-validity predicate states which model a theorem is about; it never rejects a correct proof.
11. **Assign model tiers, permissions and skills by the cost of a wrong statement.** Enforce permission boundaries by the boundary; audit allow-lists for rules that subsume denials; scope every tool’s blast radius; treat self-reports as not evidence.
12. **Audit every manuscript against its formal sources and against retrieved literature before it leaves; gate every permanent identifier behind a checklist that the verification gates do not cover.** Author of record supplied, artefact matched to the tag, metadata read against the withdrawn list, draft before publish, live record verified without credentials.
13. **Record every rule with its incident.** A rule without its reason will be shortcut; a laboratory without a lessons file repeats its month.

For a referee of an AI-assisted paper, the protocol reduces to four questions. Which tier does each claim carry, and is the gate a script or a sentence? Where is the registration commit for each prediction, and what happened to the ones that failed? What is the control for each measurement, and has it been shown to fail? And what is in the changelog of withdrawn claims — because a laboratory that has withdrawn nothing in a month of AI-assisted work has either done nothing or is not looking.

11 Limitations, and what the month did not establish

The evidence is two small laboratories, one owner, one month, and one problem area; nothing here is a controlled comparison against a laboratory run otherwise. The claim is not that the method produces more correct science than another — the month produced no theorem about fluids that the field did not have — but that it produces errors of a particular, useful shape, and the evidence for that is the table.

The method is expensive in a way that compounds. Every control needs a control; every verifier needs a verifier; the lessons file grows faster than any session can read it, and the externalised context it depends on is itself a document that can be stale. We have no principled stopping rule for the regress and used judgement, which is the thing the method exists to constrain.

Statement adequacy remains the unsolved step. Reserving it to a human is the right rule and a bottleneck; Palomar’s language-model check is a scalable filter and not a certificate; and the month’s largest failure sat exactly there. A registry entry, a ledger row and a human audit are three different things, and a laboratory needs all three.

The autoresearch pattern transferred with reservations. It chooses an implementation, never a result; its proposers must be denied the code so that checking against the code catches something; and the one screen we ran misfired on a registered rule. Its winner is a setting, and needed the ordinary controls before anything downstream cited it.

Finally, this paper was drafted by the same class of system whose failures it catalogues, from the laboratory’s own records, and it was audited by an agent following the procedure of Section 8 against those records. That is the method applied to itself and it is also a conflict of interest. The reader has the lessons file, the ledger, the changelog and the repositories, and should prefer them to any sentence here.

12 Conclusion

Formal verification has given AI-assisted mathematics a trust layer for proofs, and a registry now makes that layer public. A laboratory needs the same layer for everything that is not a proof: the choice of question, the instrument, the number, the sentence built on the number, and the act of publishing it. The method described here supplies that layer with four instruments that are each mechanical — a tiered gate, a control discipline, pre-registration, and a ledger — and an agent framework organised by the cost of a wrong statement. A month of running it in two laboratories produced no correct prediction it could not have produced otherwise, and a record of every error with its date, its cause, its catcher and its rule. Tao’s rule is to rely on the machine only where you can red-team its output. The laboratory’s job is to arrange that the red-teaming is not optional, and to write down what it found.

Disclosure

This manuscript was drafted by AI agents — Claude Fable 5.1 for the drafting and the final edits; Claude Opus 5 for the adversarial review of the laboratory report and for the manuscript audit of this paper; Claude Sonnet 5 as the independent proposers, and Opus 5 as the judge, in the screen of Section 8 — from the first laboratory’s specification, ledger, lessons file, report and design memos, and from the second laboratory’s public repository, changelog and paper, under

the adjudication of the laboratory’s human owner, who supplied the authorship, the scope and every verdict. Every external reference was retrieved and its bibliographic record confirmed before citation; the bibliography contains no entry cited from memory. Before release the draft was audited against its sources by an agent following the manuscript-auditor procedure, and the audit and the changes it produced are recorded alongside the source in the repository.

Reproducibility

The first laboratory’s repository, including the specification, ledger, lessons file, harness proposals and the report this paper draws on, is public at <https://github.com/xaviercallens/SocrateAI-Scientific-MechanicaFluidorum> (release v1.10.2 carries this paper, which is itself archived at [doi:10.5281/zenodo.22829767](https://doi.org/10.5281/zenodo.22829767); all versions resolve from [doi:10.5281/zenodo.22827933](https://doi.org/10.5281/zenodo.22827933)) and archived [15]; the second laboratory’s is public and archived [16]. The skills and agent definitions of Section 8 are in the first repository under `docs/harness/` with an installation note; the audit of this manuscript and the changes it produced are in `docs/paper/REVIEW_verifier_in_the_loop.md`.

References

- [1] T. Tao, Machine-assisted proof, *Notices Amer. Math. Soc.* **72** (2025), no. 1, DOI 10.1090/noti3041.
- [2] T. Tao, Palomar — a registry of Lean verified mathematics, blog post, 18 August 2026, <https://terrytao.wordpress.com/2026/08/18/palomar-a-registry-of-lean-verified-mathematics/>; registry at <https://palomar-registry.org/>.
- [3] Terence Tao on AI in mathematics (and beyond): a living summary compiled and drafted by an AI assistant from T. Tao’s public writing, talks and interviews, then reviewed and corrected by him; last updated 17 September 2026, <https://teorth.github.io/tao-web/ai-views.html>.
- [4] M. Bolan, T. Tao, et al., The Equational Theories Project: advancing collaborative mathematical research at scale, arXiv:2512.07087 (2025).
- [5] OpenAI, Lean certificates accompanying Navier–Stokes and Euler results, repository <https://github.com/openai/NavierStokesAndEuler>, accompanying “Finite time blowup for Navier–Stokes” and “Finite time blowup for the Euler equation”; announcement at <https://openai.com/index/navier-stokes-solution/>. Repository creation date 8 September 2026 per GitHub metadata and toolchain Lean 4.34.0-rc2 per the repository’s build instructions, both retrieved 18 September 2026.
- [6] T. Hales et al., A formal proof of the Kepler conjecture, *Forum Math. Pi* **5** (2017), e2.
- [7] K. Buzzard, Mathematical reasoning and the computer, *Bull. Amer. Math. Soc.* **61** (2024), no. 2, 211–224, DOI 10.1090/bull/1833.
- [8] T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, et al., Olympiad-level formal mathematical reasoning with reinforcement learning, *Nature* **651** (2025), 607–613, DOI 10.1038/s41586-025-09833-y.
- [9] B. Romera-Paredes et al., Mathematical discoveries from program search with large language models, *Nature* **625** (2024), 468–475.

- [10] A. Karpathy, autoresearch: AI agents running research on single-GPU nanochat training automatically, repository <https://github.com/karpathy/autoresearch> (2026).
- [11] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, *Proc. Natl. Acad. Sci. USA* **115** (2018), no. 11, 2600–2606.
- [12] P. Zgliczyński, K. Mischaikow, Rigorous numerics for partial differential equations: the Kuramoto–Sivashinsky equation, *Found. Comput. Math.* **1** (2001), 255–288, DOI 10.1007/s002080010010.
- [13] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, *CADE-28* (2021).
- [14] The mathlib Community, The Lean mathematical library, *CPP 2020*, DOI 10.1145/3372885.3373824.
- [15] Socrate AI Lab, Machine-checked triad identities for the Fourier–Galerkin truncation of the 3-D Navier–Stokes equations, with the vanishing locus of the helical interaction coefficient in closed form; with the MechanicsFluidorum repository, release v1.9.0, Zenodo, DOI 10.5281/zenodo.22823607 (2026).
- [16] X. Callens and Socrate AI Lab, The OpenAI Navier–Stokes and Euler blow-up proofs: a physical reading, not a physical refutation, version 5.6.1, Zenodo, DOI 10.5281/zenodo.22828106 (concept DOI 10.5281/zenodo.22696717, which resolves to the latest version) (2026).